الكشف عن الإنسان في الوقت الحقيقي باستعمال YOLOv5 وكاميرا الهاتف

شرح كامل لكل خطوات المشروع، مع الكود والتعليقات

0.865	37 FPS	2000	3.7 MB
mAP@0.5 على صور الاختبار	السرعة على المعالج CPU	صورة تدريب من COCO	حجم النموذج المدرَّب

ماذا يوجد في هذا الملف؟

شرح لكل مرحلة من مراحل المشروع: من جمع الصور، إلى التدريب والتقييم، إلى الكشف المباشر بكاميرا الهاتف. مع كل مرحلة ستجد الكود الحقيقي الموجود في المستودع، وتعليقاً بالعربية على كل سطر مهم، وشرحاً لماذا كُتب بهذه الطريقة.

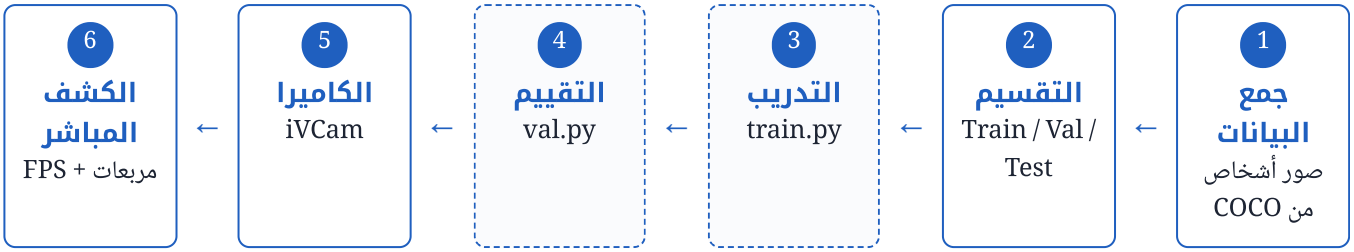
المحتويات

1. نظرة عامة على المشروع
2. كيف تعمل YOLOv5؟
3. المرحلة 1: جمع البيانات
4. المرحلة 2: تقسيم البيانات
5. المرحلة 3: التدريب
6. المرحلة 4: التقييم
7. المرحلة 5: قراءة الكاميرا
8. المرحلة 6: الكشف المباشر
9. المشاكل التي واجهتنا وحلولها
10. طريقة التشغيل على الحاسوب
11. الخطوة التالية: تجارب الأستاذ
12. أسئلة للمراجعة مع الإجابات

1. نظرة عامة على المشروع

الهدف: بناء نموذج يكتشف الإنسان (الجزء العلوي من الجسم أو الجسم كاملاً) في صور الكاميرا مباشرة، ويرسم مربعاً حوله، بسرعة تكفي للوقت الحقيقي.

المشروع سلسلة من **6 مراحل**. كل مرحلة تأخذ ناتج المرحلة التي قبلها:



المربعات ذات الخط المتقطع (3 و4) تستعمل كود مكتبة YOLOv5 الجاهز. باقي المراحل كُتب كودها في هذا المشروع.

خريطة الملفات

الملف	المرحلة	وظيفته
scripts/download_coco_person.py	1	تنزيل صور الأشخاص من COCO وتحويل العلامات إلى صيغة YOLO
scripts/split_dataset.py	2	تقسيم الصور إلى Train / Val / Test وإنشاء `data.yaml`
yolov5/train.py	3	تدريب الشبكة (مكتبة Ultralytics، لم نعدّلها)
yolov5/val.py	4	حساب Precision و Recall و mAP (مكتبة Ultralytics)
scripts/camera.py	5	قراءة كاميرا الهاتف في خيط منفصل حتى لا يتأخر الفيديو
scripts/list_cameras.py	5	إيجاد رقم كاميرا الهاتف (iVCam)
scripts/detect_realtime.py	6	تحميل النموذج، الكشف، رسم المربعات، حساب FPS
scripts/capture_images.py	إضافي	التقاط صور خاصة بك من الهاتف (إذا أردت إضافة بياناتك)
models/person_yolov5n.pt	الناتج	النموذج المدرَّب (الأوزان)
results/	الناتج	منحنيات التدريب، صور الاختبار، فيديو تجريبي

تمييز مهم: ما كتبناه، وما استعملناه جاهزاً

مجلد `yolov5/` هو مكتبة شركة Ultralytics. فيها بنية الشبكة وخوارزمية التدريب والتقييم، ولم نعدّل فيها شيئاً. ما كتبناه نحن هو تجهيز البيانات (المرحلتان 1 و2)، وقراءة الكاميرا والكشف المباشر (المرحلتان 5 و6)، والأوامر التي تشغّل التدريب والتقييم.

2. كيف تعمل YOLOv5؟

الفكرة: "You Only Look Once"

الطرق القديمة (مثل R-CNN) تمر على الصورة مرتين: مرة تقترح أماكن مرشحة، ومرة تصنّف كل مكان، وهذا بطيء. **YOLO تنظر إلى الصورة مرة واحدة فقط**، وتتنبأ بكل المربعات في نفس الوقت. لهذا هي سريعة وتصلح للوقت الحقيقي.

الشبكة (Grid) والمقاييس الثلاثة

تقسم YOLO الصورة إلى خلايا، وكل خلية مسؤولة عن الأجسام التي يقع **مركزها** داخلها. YOLOv5 تستعمل 3 شبكات بأحجام مختلفة. بصورة 416×416:

المقياس	حجم الشبكة	كل خلية تغطي	مناسب لـ
P3	52 × 52	8 × 8 px	الأجسام الصغيرة (شخص بعيد)
P4	26 × 26	16 × 16 px	الأجسام المتوسطة
P5	13 × 13	32 × 32 px	الأجسام الكبيرة (شخص قريب من الكاميرا)

المربعات المرجعية (Anchors)

كل خلية تملك 3 مربعات بأشكال جاهزة (مثلاً مربع طويل ونحيف يناسب شخصاً واقفاً). النموذج لا يرسم المربع من الصفر، بل **يعدّل** أقرب مربع مرجعي: يزيحه ويكبّره أو يصغّره.

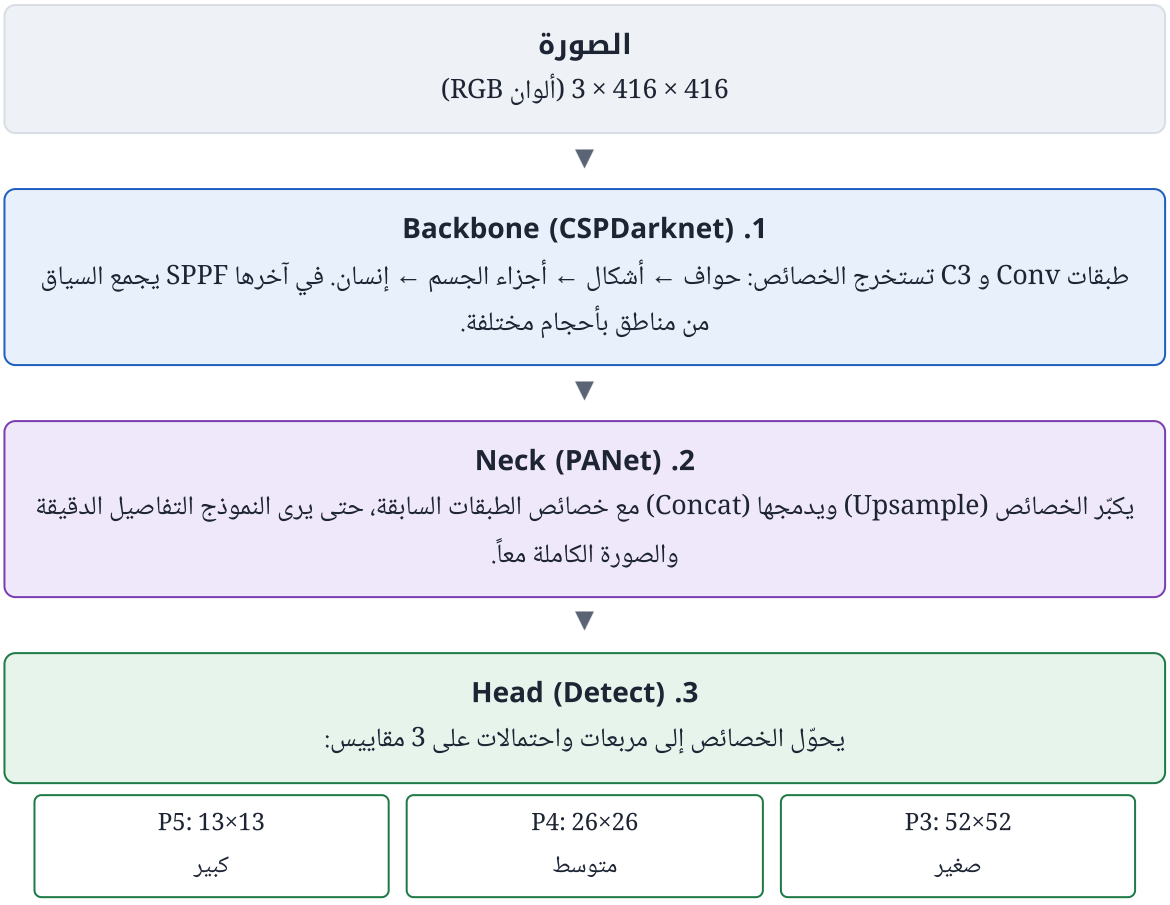
ماذا تُخرج الشبكة؟

لكل مربع مرجعي 6 أرقام: `[x, y, w, h, objectness, class]`

- **x, y, w, h**: مكان المربع وحجمه.
- **objectness**: احتمال وجود جسم هنا أصلاً.
- **class**: احتمال أن يكون الجسم شخصاً (عندنا فئة واحدة).

> عدد التنبؤات في كل صورة: **10,647** = 3 × (13×13 + 26×26 + 52×52) مربعاً مرشحاً. معظمها يُحذف في النهاية، ويبقى مربع واحد لكل شخص.

بنية الشبكة: ثلاثة أجزاء



وظيفة كل نوع من الطبقات

الطبقة	وظيفتها
Conv	التفاف + BatchNorm + دالة تفعيل SiLU. بخطوة 2 (stride=2) تصغّر الصورة للنصف وتزيد عدد الخصائص.
C3 (CSP)	تقسم الخصائص إلى مسارين، أحدهما يمر في كتل Bottleneck فيها وصلات بقايا (Residual). هذا يقلل الحساب ويساعد التدرّج على المرور في الشبكة العميقة.
SPPF	Max-Pooling متكرر بأحجام مختلفة، فيرى النموذج السياق حول الجسم.
Upsample + Concat	تكبير الخصائص العميقة ودمجها مع الخصائص السطحية (Neck).
Detect	الرأس النهائي: يُخرج المربعات على المقاييس الثلاثة.

أحجام YOLOv5

النموذج	depth_multiple	width_multiple	المعاملات	الاستعمال
yolov5n (نموذجنا)	0.33	0.25	1.76 M	الأسرع، مناسب لـ CPU
yolov5s	0.33	0.50	7.2 M	سريع وأدق
yolov5m	0.67	0.75	21 M	متوسط
yolov5l	1.0	1.0	46 M	دقة أعلى، أبطأ

كل الأحجام لها **نفس البنية**. الفرق فقط في عمق الشبكة (عدد تكرارات C3) وعرضها (عدد الفلاتر في كل طبقة).

3. المرحلة 1: جمع البيانات

`scripts/download_coco_person.py`

المشكلة: النموذج يحتاج مئات الصور لأشخاص، وفي كل صورة مربع مرسوم حول كل شخص. رسم هذه المربعات باليد يأخذ أياماً.

الحل: مجموعة البيانات العامة **COCO 2017** فيها أكثر من 118 ألف صورة، رسم الباحثون عليها المربعات مسبقاً لـ 80 نوعاً من الأشياء. نأخذ منها صور الأشخاص فقط.

أ) تنزيل ملف العلامات

```
ANN_URL = "http://images.cocodataset.org/annotations/annotations_trainval2017.zip"
PERSON_ID = 1           # 1 هو "person" في COCO رقم فئة

# ملف JSON كبير فيه وصف كل صورة: عنوانها، أبعادها، والمربعات الموجودة فيها
urllib.request.urlretrieve(ANN_URL, zip_path)
with open(ann_json) as f:
    coco = json.load(f)
```

ب) اختيار الصور المناسبة

```
def select_images(coco, num, min_area, seed=0):
    images = {im["id"]: im for im in coco["images"]}    # قاموس: رقم الصورة ← معلوماتها
    boxes, skip = {}, set()
    for ann in coco["annotations"]:                     # نمر على كل مربع في COCO
        if ann["category_id"] != PERSON_ID:
            continue                                    # ليس إنساناً (سيارة، قطة...) ← تجاهله
        if ann.get("iscrowd"):
            skip.add(ann["image_id"])                   # صورة حشد ← نستبعد الصورة كلها
            continue
        im = images[ann["image_id"]]
        if ann["bbox"][2] * ann["bbox"][3] < min_area * im["width"] * im["height"]:
            continue                                    # الشخص أصغر من 2% من الصورة ← تجاهله
        boxes.setdefault(ann["image_id"], []).append(ann["bbox"])
    ids = sorted(i for i in boxes if i not in skip)
    random.Random(seed).shuffle(ids)                    # خلط عشوائي ثابت (نفس النتيجة كل مرة)
    return [(images[i], boxes[i]) for i in ids[:num]]   # أول num صورة
```

لماذا هذه الشروط؟

- **صور الحشود (iscrowd):** فيها مربع واحد كبير حول مجموعة أشخاص، وهذا يعلّم النموذج شيئاً خاطئاً.
- **الأشخاص الأصغر من 2%:** بضعة بكسلات فقط، فيصعب التعلم منهم. لكن لهذا القرار ثمن: النموذج أصبح أضعف في كشف الأشخاص البعيدين جداً (رأينا ذلك في فيديو المشاة).

ج) تحويل المربع من صيغة COCO إلى صيغة YOLO

```
def coco_to_yolo(bbox, img_w, img_h):
    # COCO: [x_min, y_min, العرض, الارتفاع] بالبكسل، من الزاوية العلوية اليسرى
    x, y, w, h = bbox
    # YOLO: (المركز x، المركز y، العرض، الارتفاع)، كلها مقسومة على أبعاد الصورة (بين 0 و 1)
    return (x + w / 2) / img_w, (y + h / 2) / img_h, w / img_w, h / img_h
```

مثال

صورة 400×600، فيها مربع COCO `[100, 50, 200, 300]` :

المركز x = (100 + 100) / 400 = 0.5 ، المركز y = (50 + 150) / 600 = 0.333 ، العرض = 200/400 = 0.5 ،

الارتفاع = 300/600 = 0.5

السطر في ملف YOLO: `0 0.5 0.333 0.5 0.5` (الرقم الأول 0 = فئة person)

لماذا أرقام نسبية؟ لأن YOLO تغيّر حجم الصورة (مثلاً إلى 416×416)، والأرقام النسبية تبقى صحيحة مهما كان الحجم.

د) تنزيل الصور وكتابة ملفات العلامات

```
for n, (im, bboxes) in enumerate(selected, 1):
    stem = f"coco_{im['id']:012d}"                      # اسم الملف مثل coco_000000166696
    urllib.request.urlretrieve(im["coco_url"], img_path)  # تنزيل الصورة
    with open(os.path.join(lbl_dir, stem + ".txt"), "w") as f:
        for b in bboxes:                                # سطر لكل شخص في الصورة
            f.write("0 %.6f %.6f %.6f %.6f\n" % coco_to_yolo(b, im["width"],
im["height"]))
```

النتيجة

2000 صورة في `data/raw/images/`، ولكل صورة ملف علامات بنفس الاسم في `data/raw/labels/`. المجموع 4051 شخصاً.

الأمر الذي نفّذناه:

```
python scripts/download_coco_person.py --num 2000 --out data/raw --split train2017 --min-
area 0.02
```

4. المرحلة 2: تقسيم البيانات

`scripts/split_dataset.py`

لماذا نقسم البيانات؟

إذا اختبرنا النموذج على نفس الصور التي تدرّب عليها، فهذا مثل امتحان يعرف الطالب أسئلته مسبقاً: النتيجة ممتازة لكنها كاذبة.

لذلك نقسم الصور 3 أقسام:

القسم	العدد	الدور	تشبيه
Train	1600 (80%)	النموذج يتعلم منها ويعدّل أوزانه	الدروس
Validation	200 (10%)	نختبر بعد كل جولة لنختار أفضل نسخة من النموذج	التمارين
Test	200 (10%)	اختبار نهائي واحد على صور لم يرها النموذج أبداً	الامتحان النهائي

أ) إيجاد كل صورة مع ملف علاماتها

```
def find_pairs(src):
    pairs, missing = [], 0
    for name in sorted(os.listdir(img_dir)):
        stem, ext = os.path.splitext(name)              # coco_123.jpg" ← ("coco_123", ".jpg")"
        if ext.lower() not in IMG_EXT:
            continue                                    # ليس صورة ← تجاهله
        lbl = os.path.join(lbl_dir, stem + ".txt")
        if os.path.exists(lbl):
            pairs.append((os.path.join(img_dir, name), lbl))   # زوج: (صورة، علامات)
        else:
            missing += 1                                # صورة بلا علامات ← لا نستعملها
    return pairs, missing
```

ب) الخلط والتقسيم

```
def split(pairs, train, val, seed):
    pairs = pairs[:]
    random.Random(seed).shuffle(pairs)          # خلط عشوائي، لكن ثابت بفضل seed=42
    n_train = int(len(pairs) * train)           # 1600 = 0.8 × 2000
    n_val = int(len(pairs) * val)               # 200 = 0.1 × 2000
    return {
        "train": pairs[:n_train],
        "val": pairs[n_train:n_train + n_val],
        "test": pairs[n_train + n_val:],        # الباقي = 200
    }
```

لماذا الخلط؟ ولماذا seed؟

الخلط: حتى لا تقع كل الصور المتشابهة في قسم واحد.

seed=42: حتى يعطي التقسيم نفس النتيجة كل مرة. هذا مهم جداً لتجارب الأستاذ القادمة: يجب أن تُختبر كل التجارب على نفس صور الاختبار، وإلا لن تكون المقارنة عادلة.

ج) إنشاء ملف data.yaml

```
with open(yaml_path, "w") as f:
    f.write(f"path: {os.path.abspath(args.dst)}\n")
    f.write("train: images/train\nval: images/val\ntest: images/test\n")
    f.write(f"nc: {len(args.names)}\n")              # عدد الفئات = 1
    f.write("names: [" + ", ".join(f"'{n}'" for n in args.names) + "]\n")
```

الملف الناتج يخبر YOLOv5 أين الصور وما أسماء الفئات:

```
path: data/dataset
train: images/train
val: images/val
test: images/test
nc: 1                  # فئة واحدة
names: ['person']
```

الأمر الذي نفّذناه:

```
python scripts/split_dataset.py --src data/raw --dst data/dataset --names person
```

5. المرحلة 3: التدريب

`yolov5/train.py`

الأمر الذي نفّذناه

```
python train.py --img 416 --batch 32 --epochs 40 \
                --data data/dataset/data.yaml --weights yolov5n.pt \
                --device cpu --workers 4 --name person
```

الخيار	القيمة	المعنى
--img	416	كل صورة تُصغَّر إلى 416×416. أصغر من 640 الافتراضي، فهو أسرع على CPU.
--batch	32	عدد الصور التي تُعالج معاً في كل خطوة تعديل.
--epochs	40	عدد المرات التي يمر فيها النموذج على كل صور التدريب.
--data	data.yaml	ملف البيانات الذي أنشأناه في المرحلة 2.
--weights	yolov5n.pt	**نبدأ من نموذج مدرَّب مسبقاً** (التعلم بالنقل).
--device	cpu	التدريب على المعالج (لا يوجد GPU).

التعلم بالنقل (Transfer Learning)

لم نبدأ من الصفر. الملف `yolov5n.pt` نموذج تعلّم مسبقاً على COCO كاملة (80 نوعاً). YOLOv5 تحذف رأس الكشف القديم (80 فئة) وتضع رأساً جديداً لفئة واحدة، ويبقى الـ Backbone كما هو لأنه يعرف أصلاً كيف يرى الحواف والأشكال والأجسام. **لهذا وصلنا إلى mAP = 0.69 بعد 3 جولات فقط.**

فحص المربعات المرجعية (AutoAnchor)

```
AutoAnchor: 4.77 anchors/target, 1.000 Best Possible Recall (BPR). Current anchors are a
good fit ✅
```

معناه أن المربعات المرجعية الافتراضية تناسب أشكال الأشخاص في بياناتنا، فلا حاجة لتغييرها.

ماذا يحدث داخل train.py؟

```
# (شرح مبسّط لما تفعله المكتبة، وليس الكود الحرفي)
for epoch in range(40):                          # 40 جولة
    for images, targets in train_loader:         # 1600 / 32 = 50 دفعة في كل جولة
        images = augment(images)                 # تكبير البيانات: Mosaic، ألوان، قلب...
        preds = model(images)                    # تمرير الصور في الشبكة ← 10,647 مربعاً لكل صورة
        loss = box_loss + obj_loss + cls_loss    # حساب مقدار الخطأ
        loss.backward()                          # Backpropagation: اتجاه تعديل كل وزن
        optimizer.step()                         # SGD: تعديل الأوزان بخطوة حجمها learning rate
    mAP = validate(model, val_loader)            # اختبار على 200 صورة Validation
    if mAP > best:
        save("best.pt")                          # نحفظ أفضل نسخة فقط
```

دالة الخسارة (Loss): مقدار الخطأ

النوع	ماذا يقيس	في تدريبنا
box_loss	هل المربع في المكان الصحيح؟ (يُقاس بـ CIoU)	✅ 0.087 → 0.029
obj_loss	هل عرف أين يوجد جسم وأين لا يوجد؟	✅ 0.033 → 0.022
cls_loss	هل صنّف نوع الجسم صحيحاً؟	**0 دائماً**، لأن عندنا فئة واحدة فقط

تكبير البيانات (Data Augmentation)

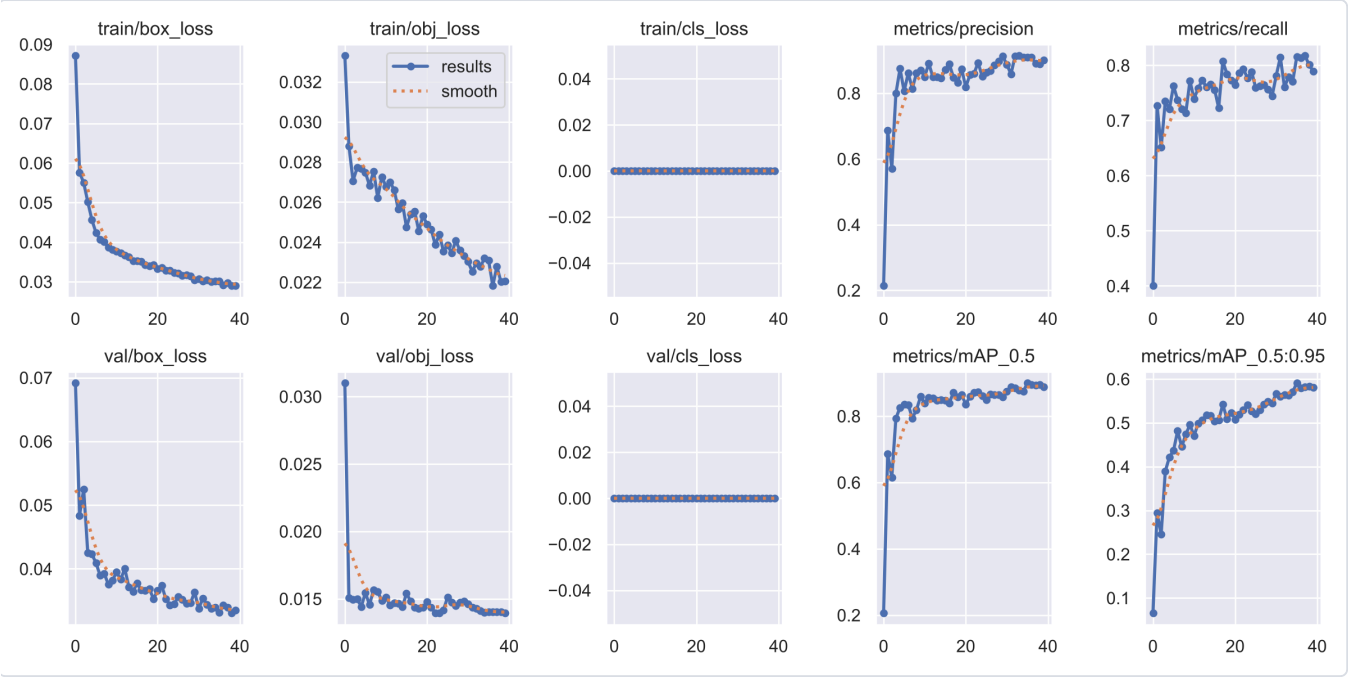
YOLOv5 تغيّر الصور تلقائياً أثناء التدريب، حتى لا يحفظ النموذج الصور بل يفهمها:

- **Mosaic:** دمج 4 صور في صورة واحدة. أهم حيلة في YOLOv5.
- **HSV:** تغيير اللون والتشبع والإضاءة.
- **fliplr:** قلب الصورة أفقياً (احتمال 50%).
- **scale / translate:** تكبير أو تصغير وإزاحة.

تطوّر النتيجة أثناء التدريب

الجولة	1	3	10	18	36 (الأفضل)	40
mAP@0.5 على Validation	0.205	0.687	0.859	0.872	**0.899**	0.888

لاحظ أن النتيجة **ترتفع وتنخفض** بين الجولات، وهذا طبيعي. لهذا نحفظ `best.pt` (أفضل جولة) وليس آخر جولة فقط. مدة التدريب: **65 دقيقة** على CPU بـ 4 أنوية.



منحنيات التدريب: الخسارة (Loss) تنخفض، و Precision و Recall و mAP ترتفع. الرسم في الأسفل في الوسط (val/cls_loss) يساوي صفراً لأن عندنا فئة واحدة.

6. المرحلة 4: التقييم

`yolov5/val.py`

```
python val.py --weights runs/train/person/weights/best.pt \
              --data data/dataset/data.yaml --task test --img 416
# --task test ← اختبر على صور Test التي لم يرها النموذج أبداً
```

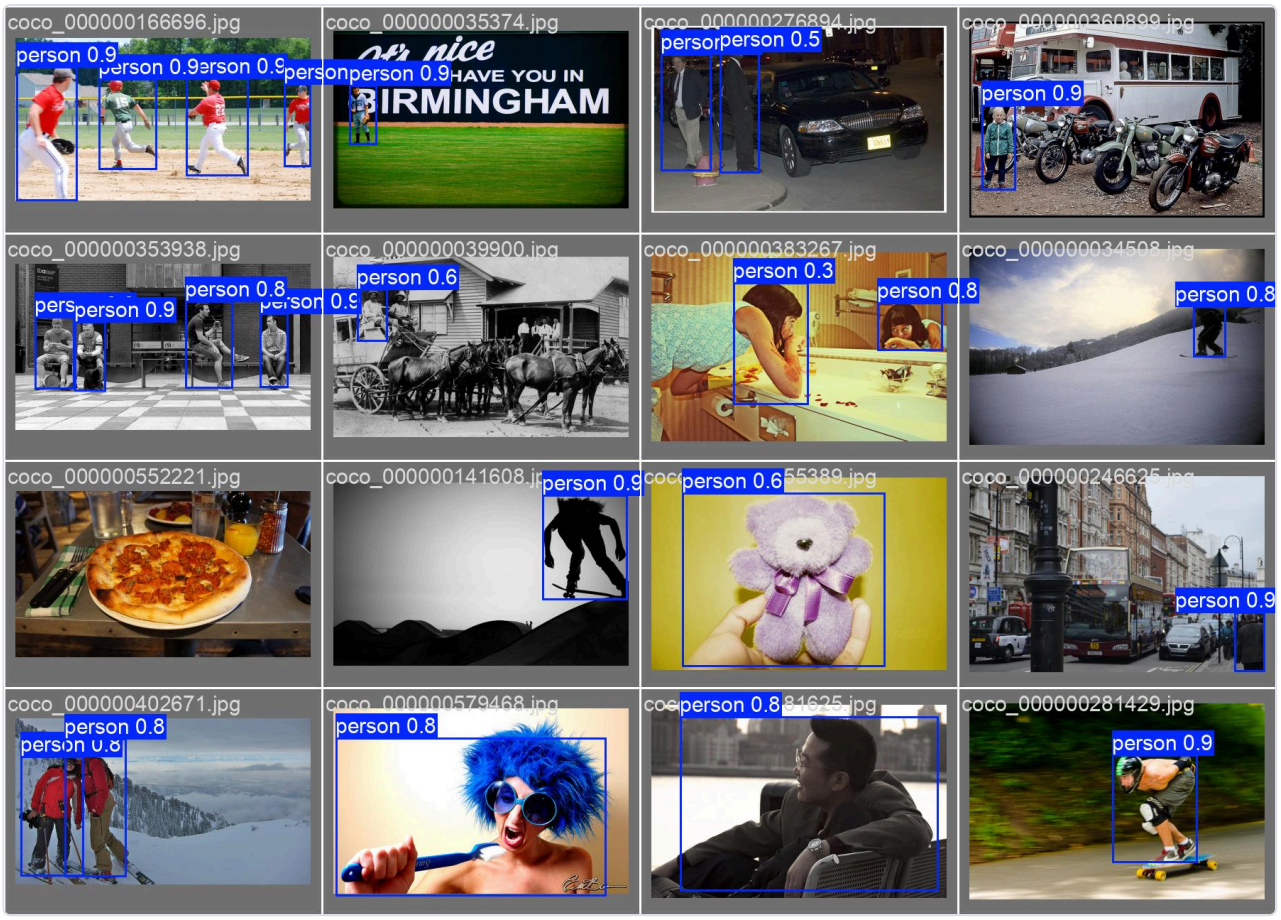
IoU: كيف نعرف أن المربع صحيح؟

IoU (Intersection over Union) = مساحة التقاطع بين مربع النموذج والمربع الصحيح ÷ مساحة اتحادهما. إذا كانت النتيجة 1 فالمربعان متطابقان، وإذا كانت 0 فهما لا يتلامسان. نعتبر المربع **صحيحاً** إذا كان $IoU \geq 0.5$.

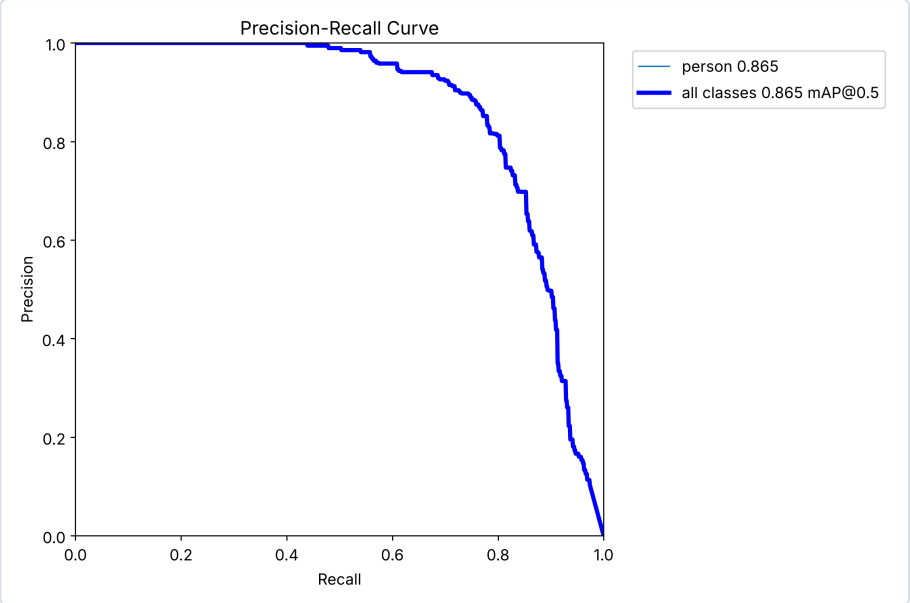
النتائج النهائية على صور الاختبار

المقياس	النتيجة	المعنى
Precision	0.870	من كل 100 مربع رسمه النموذج، 87 فيها شخص حقيقي
Recall	0.764	من كل 100 شخص موجود فعلاً، اكتشف النموذج 76
mAP@0.5	**0.865**	الدقة العامة، عندما نقبل المربع إذا كان $IoU \geq 0.5$
mAP@0.5:0.95	0.571	مقياس أصعب: متوسط الدقة عندما نشترط IoU من 0.5 حتى 0.95
سرعة الاستدلال	22.7 ms	لكل صورة على CPU (دفعة 32)

> **لماذا نتيجة Test (0.865) أقل من Validation (0.899)؟**
> لأننا اخترنا أفضل جولة بناءً على Validation، فنتيجتها متفائلة قليلاً. نتيجة Test هي **الرقم الصادق** الذي يُكتب في التقرير.



أمثلة كشف على صور الاختبار. النموذج يكتشف الأشخاص في أوضاع مختلفة: يمشون، يجلسون، يتزلجون، وجوه قريبة. لاحظ خطأً محتملاً: مربع حول دبّ بنفسجي ويد بثقة 0.6.



منحنى Precision–Recall: كلما اقترب المنحنى من الزاوية العليا اليمنى كان النموذج أفضل. المساحة تحته = 0.865 = mAP@0.5.

7. المرحلة 5: قراءة الكاميرا

`scripts/camera.py`

المشكلة

لو قرأنا صورة من الكاميرا، ثم كشفنا فيها، ثم قرأنا الصورة التالية، فإن الصور تتراكم في الذاكرة أثناء الكشف، ويصبح الفيديو **متأخراً** عن الواقع بعدة ثوانٍ.

الحل: خيط (Thread) منفصل للقراءة

```
class VideoStream:
    def __init__(self, source):
        self.source = parse_source(source)        # URL أو عنوان ،(كاميرا) 1 رقم ← "1"
        self.cap = open_capture(self.source)      # فتح الكاميرا
        if not self.cap.isOpened():
            raise RuntimeError("Cannot open camera source")
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # ذاكرة صورة واحدة فقط
        self.ok, self.frame = self.cap.read()
        self.lock = threading.Lock()              # قفل: يمنع الخيطين من لمس الصورة في نفس اللحظة
        self.stopped = False
        self.thread = threading.Thread(target=self._update, daemon=True)
        self.thread.start()                       # يبدأ القراءة في الخلفية

    def _update(self):                            # يعمل في الخلفية بلا توقف
        while not self.stopped:
            ok, frame = self.cap.read()
            if not ok:
                time.sleep(0.05)                  # انقطاع قصير في الشبكة ← حاول مرة أخرى
                continue
            with self.lock:
                self.ok, self.frame = ok, frame   # نحتفظ دائماً بآخر صورة فقط

    def read(self):                               # البرنامج الرئيسي يأخذ أحدث صورة
        with self.lock:
            return self.ok, self.frame.copy()
```

الفكرة

خيط يقرأ من الكاميرا باستمرار، وخيط آخر يكشف. الكشف يأخذ دائماً **أحدث** صورة، والصور القديمة تُستبدل، فلا يحدث تأخير.

دعم iVCam على Windows

```
def open_capture(source):
    # الكاميرات الافتراضية (iVCam، OBS) تعمل على Windows بشكل أفضل مع DirectShow
    if isinstance(source, int) and sys.platform.startswith("win"):
        cap = cv2.VideoCapture(source, cv2.CAP_DSHOW)
        if cap.isOpened():
            return cap
    return cv2.VideoCapture(source)              # الطريقة العادية لباقي الحالات
```

إيجاد رقم كاميرا الهاتف

`scripts/list_cameras.py`

```
for i in range(6):                               # جرّب الكاميرات من 0 إلى 5
    cap = open_capture(i)
    ok, frame = cap.read() if cap.isOpened() else (False, None)
    cap.release()
    if ok and frame is not None:
        cv2.imwrite(f"camera_{i}.jpg", frame)    # احفظ صورة لتعرف أي كاميرا هي الهاتف
        print(f"camera {i}: OK")
```

على حاسوبك ظهرت الكاميرا 0 (كاميرا الحاسوب) والكاميرا **1 (الهاتف عبر iVCam)**.

8. المرحلة 6: الكشف المباشر

`scripts/detect_realtime.py`

أ) تحميل النموذج

```
def load_model(weights, conf, device, yolov5_dir=""):
    local = find_local_yolov5(yolov5_dir)        # ابحث عن مجلد yolov5 على الحاسوب
    if local:
        # يقرأ بنية الشبكة من مجلد yolov5، ويضع فيها الأوزان التي تعلّمناها
        model = torch.hub.load(local, "custom", path=weights, device=device,
source="local")
    else:
        model = torch.hub.load("ultralytics/yolov5", "custom", path=weights,
device=device)
    model.conf = conf                            # حد الثقة: نحذف المربعات التي ثقتها أقل من 0.4
    return model
```

ب) الكشف في صورة واحدة

```
def detect(model, frame, img_size):
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)        # OpenCV يقرأ BGR، والنموذج تعلّم على RGB
    return model(rgb, size=img_size).xyxy[0].cpu()      # لكل شخص: [x1, y1, x2, y2, الثقة, الفئة]
```

> **لماذا تحويل RGB ← BGR مهم؟ (جرّبناه فعلاً)**
>
> على 20 صورة اختبار فيها 40 شخصاً: **بدون التحويل** اكتشف النموذج 30 شخصاً فقط، و**مع التحويل** اكتشف 41.

داخل `model(...)` تحدث تلقائياً كل خطوات YOLO:

1. تصغير الصورة إلى `img_size` مع الحفاظ على النسبة.
2. تمرير الصورة في الشبكة ← 10,647 مربعاً مرشحاً.
3. حذف المربعات التي ثقتها أقل من `conf` (0.4).
4. **NMS (Non-Maximum Suppression):** إذا غطّت عدة مربعات نفس الشخص (IoU بينها > 0.45)، يبقى أقواها فقط.
5. إرجاع الإحداثيات إلى حجم الصورة الأصلي.

ج) رسم المربعات

```
def draw(frame, det, names):
    for *xyxy, conf, cls in det.tolist():          # لكل شخص مكتشف
        x1, y1, x2, y2 = map(int, xyxy)            # زاويتا المربع (أعلى يسار، أسفل يمين)
        label = f"{names[int(cls)]} {conf:.2f}"    # النص: "person 0.87"
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)    # مربع أخضر بسُمك 2
        cv2.putText(frame, label, (x1, max(y1 - 8, 15)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
    return frame
```

د) الحلقة الرئيسية وحساب FPS

```
while True:
    ok, frame = stream.read()                      # أحدث صورة من الهاتف
    det = detect(model, frame, args.img_size)      # الكشف
    draw(frame, det, model.names)                  # الرسم

    now = time.time()
    inst = 1.0 / max(now - prev, 1e-6)             # الزمن بين صورتين ← عدد الصور في الثانية
    fps = inst if fps == 0 else 0.9 * fps + 0.1 * inst    # تنعيم: حتى لا يقفز الرقم على الشاشة
    prev = now
    cv2.putText(frame, f"FPS: {fps:.1f}   objects: {len(det)}", (10, 30), ...)
    cv2.imshow("YOLOv5 real-time", frame)          # عرض النتيجة

    key = cv2.waitKey(1) & 0xFF
    if key == ord("q"):  break                     # خروج ← q
    if key == ord("s"):  cv2.imwrite(path, frame)  # صورة شاشة للتقرير ← s
```



لقطة من التجربة على فيديو مشاة: 34.9 FPS على CPU. النموذج اكتشف الشخص القريب على اليمين (person 0.4)، لكنه لم يكتشف الشخصين البعيدين الصغيرين. هذه نقطة الضعف المعروفة (انظر الفصل 9).

حجم الصورة / حد الثقة	متوسط الأشخاص المكتشفين في كل لقطة	FPS
img 416, conf 0.40	0.23	41.9
img 416, conf 0.25	0.25	44.6
img 640, conf 0.25	0.70	26.3

هذه أول تجربة على معامل واحد: **تكبير الصورة إلى 640 يضاعف كشف الأشخاص البعيدين ثلاث مرات تقريباً، لكن السرعة تنخفض**. هذه هي المقايضة بين الدقة والسرعة.

9. المشاكل التي واجهتنا وحلولها

هذا الفصل مهم للتقرير، لأنه يبيّن أنك فهمت ما حدث وليس فقط النتيجة.

المشكلة	السبب	الحل
`pip` غير معروف في PowerShell	Python لم يكن مثبّتاً على الحاسوب	تثبيت Python 3.12 مع خيار "Add to PATH"
`scipy not found`	مكتبة يحتاجها YOLOv5 كانت ناقصة في `requirements.txt`	إضافتها إلى الملف
`No module named 'pathlib._local'`	النموذج حُفظ على Python 3.13، ويحتوي على كائنات `PosixPath` لا توجد إلا في 3.13. الحاسوب يعمل بـ 3.12	إعادة حفظ النموذج بالأوزان فقط، بدون هذه الكائنات، ثم التأكد أنه يعمل على Python 3.12
`HTTP Error 400`	`torch.hub` حاول تنزيل كود YOLOv5 من GitHub وفشل	تحميل YOLOv5 من المجلد المحلي `yolov5/`
الأشخاص البعيدون لا يُكتشفون	استبعدنا الأشخاص الأصغر من 2% من التدريب، وحجم 416 فيه تفاصيل أقل	استعمال `--img-size 640`، أو إعادة التدريب مع الأشخاص الصغار
الملفات غير موجودة عند التشغيل	PowerShell كان في المجلد الأعلى، وليس في مجلد المشروع	الدخول إلى المجلد أولاً بـ `cd`

نقطة ضعف النموذج الحالي

النموذج ممتاز للأشخاص **القريبين** من الكاميرا (الجزء العلوي أو الجسم كاملاً)، وهذا هو وضع كاميرا الهاتف. لكنه ضعيف مع الأشخاص **البعيدين جداً** (مثل كاميرا مراقبة عالية). السبب قرار اتخذناه في المرحلة 1 (استبعاد الأشخاص الأصغر من 2%).

10. طريقة التشغيل على الحاسوب (Windows)

مرة واحدة فقط: التثبيت

```
# 1) تثبيت Python 3.12 (مع خيار Add python.exe to PATH)
winget install -e --id Python.Python.3.12

# 2) تنزيل المشروع ومكتبة YOLOv5
git clone -b claude/intelligent-mayer-pvt3yy https://github.com/chemsoubensalah/real_time-_object_detection.git
cd real_time-_object_detection
git clone https://github.com/ultralytics/yolov5

# 3) تثبيت المكتبات
python -m pip install -r requirements.txt
```

كل مرة: التشغيل

```
# 1) افتح iVCam على الهاتف وعلى الحاسوب

# 2) ادخل إلى مجلد المشروع (علامات التنصيص ضرورية بسبب المسافات)
cd "C:\Users\chems\real time humain detection\real_time-_object_detection"

# 3) اعرف رقم كاميرا الهاتف (عندك = 1)
python scripts/list_cameras.py

# 4) الكشف المباشر
python scripts/detect_realtime.py --weights models/person_yolov5n.pt --source 1

# خيارات إضافية:
#   --conf 0.25          حد ثقة أقل ← يكتشف أكثر، لكن أخطاء أكثر
#   --img-size 640       دقة أعلى للأشخاص البعيدين، سرعة أقل
#   --save-video demo.mp4   تسجيل فيديو للأستاذ
```

الزر	الوظيفة
s	حفظ صورة شاشة في `runs/realtime/`
q	الخروج وطباعة متوسط FPS

11. الخطوة التالية: تجارب الأستاذ

طلب الأستاذ تغيير المعاملات (مثل learning rate) وبنية الشبكة، وفهم أثر كل تغيير. القاعدة: **غيّر متغيّراً واحداً فقط في كل تجربة**، وأبقِ الباقي كما هو (نفس البيانات، نفس عدد الجولات، نفس seed).

أين تتغيّر هذه الإعدادات؟

المكان	ماذا فيه
yolov5/data/hyps/hyp.scratch-low.yaml	معاملات التدريب: `lr0`، `momentum`، `weight_decay`، أوزان الخسارة، تكبير البيانات
yolov5/models/yolov5n.yaml	بنية الشبكة: العمق والعرض، المربعات المرجعية، الطبقات وترتيبها
train.py (خيارات الأمر)	`--epochs`، `--batch`، `--img`، `--optimizer`، `--weights`
yolov5/models/common.py	دالة التفعيل: السطر `default_act = nn.SiLU()`

أهم سطور ملف المعاملات

```
lr0: 0.01              # معدل التعلم في البداية: حجم خطوة تعديل الأوزان
lrf: 0.01              # معدل التعلم في النهاية = lr0 × lrf
momentum: 0.937        # الزخم: يجعل الخطوات أكثر استقراراً
weight_decay: 0.0005   # يعاقب الأوزان الكبيرة ← يقلل الحفظ (overfitting)
warmup_epochs: 3.0     # أول 3 جولات يبدأ معدل التعلم صغيراً ثم يرتفع
box: 0.05              # أهمية خطأ مكان المربع
obj: 1.0               # أهمية خطأ "هل يوجد جسم؟"
fliplr: 0.5            # احتمال قلب الصورة أفقياً
mosaic: 1.0            # احتمال دمج 4 صور
```

أهم سطور ملف البنية

```
depth_multiple: 0.33  # عمق الشبكة (عدد تكرارات C3)
width_multiple: 0.25  # عرض الشبكة (عدد الفلاتر)
backbone:
  [[-1, 1, Conv, [64, 6, 2, 2]],    # [من أين, التكرار, نوع الطبقة, المعاملات]
   [-1, 1, Conv, [128, 3, 2]],      # 128 فلتر، حجم 3، خطوة 2 ← تصغير للنصف
   [-1, 3, C3, [128]],              # كتلة C3 مكررة 3 مرات
   ...
   [-1, 1, SPPF, [1024, 5]]]        # تجميع السياق
```

التجارب المقترحة

#	التجربة	القيم	ماذا نتعلم
1	Learning rate	0.001 / 0.01 / 0.1	صغير جداً ← بطيء، كبير جداً ← غير مستقر
2	Batch size	8 / 16 / 32	استقرار التدريب واستهلاك الذاكرة
3	حجم الصورة	320 / 416 / 640	الدقة مقابل السرعة
4	المُحسِّن	SGD / Adam / AdamW	سرعة التعلم والدقة النهائية
5	مدرَّب مسبقاً أو من الصفر	yolov5n.pt / ''	قيمة التعلم بالنقل
6	Mosaic	1.0 / 0.0	أهمية تكبير البيانات
7	العمق والعرض	n / s / m	حجم الشبكة مقابل السرعة
8	حذف SPPF	—	فائدة تجميع السياق
9	Conv ← C3 عادي	—	فائدة CSP والوصلات المتبقية
10	حذف الرأس P3	—	أهمية الكشف على عدة مقاييس (الأجسام الصغيرة)
11	دالة التفعيل	SiLU / ReLU / LeakyReLU	أثر دالة التفعيل
12	جيل آخر من YOLO	YOLOv8n / YOLO11n	مقارنة الأجيال (بدون Anchors)

نصيحة

لا تعدّل الملفات الأصلية. انسخ الملف بإسم جديد (مثل `hyp_lr0.001.yaml`)، عدّل النسخة، ثم مرّرها: `python train.py --hyp hyp_lr0.001.yaml --cfg yolov5n_noSPPF.yaml ...`

12. أسئلة للمراجعة مع الإجابات

حاول أن تجيب بنفسك أولاً، ثم اقرأ الإجابة.

1. لماذا نقسم البيانات إلى Train و Val و Test، وليس إلى Train و Test فقط؟

لأننا نستعمل Val أثناء التدريب لاختيار أفضل جولة. هذا الاختيار يجعل نتيجة Val متفائلة. نحتاج Test منفصلاً لم يُستعمل في أي قرار، حتى نحصل على تقييم صادق.

2. ما الفرق بين صيغة المربع في COCO وفي YOLO؟

COCO: الزاوية العلوية اليسرى + العرض + الارتفاع، بالبكسل. YOLO: المركز + العرض + الارتفاع، مقسومة على أبعاد الصورة (بين 0 و 1).

3. لماذا بدأنا من yolov5n.pt ولم نبدأ من الصفر؟

التعلم بالنقل: الـ Backbone تعلّم مسبقاً الحواف والأشكال من 118 ألف صورة. نحتاج فقط أن يتخصص في الأشخاص، فنصل إلى دقة عالية بسرعة وبصور قليلة.

4. لماذا cls_loss يساوي صفراً دائماً في تدريبنا؟

لأن عندنا فئة واحدة فقط (person). لا يوجد شيء نخطئ في تصنيفه.

5. ما وظيفة NMS؟

الشبكة تُخرج عدة مربعات متداخلة حول نفس الشخص. NMS يبقي المربع الأعلى ثقة، ويحذف المربعات التي يزيد IoU بينها وبينه على 0.45.

6. لماذا استعملنا Thread في قراءة الكاميرا؟

حتى لا تتراكم الصور أثناء الكشف. الخيط يقرأ باستمرار ويحتفظ بآخر صورة فقط، فلا يتأخر الفيديو عن الواقع.

7. لماذا نتيجة Test أقل من نتيجة Validation؟

لأن best.pt اختير لأنه الأفضل على Val، فنتيجته عليه متفائلة. Test صور جديدة تماماً.

8. ماذا يحدث لو رفعنا --conf إلى 0.8؟ ولو خفّضناه إلى 0.1؟

0.8: مربعات أقل لكنها أدق (Precision أعلى، Recall أقل). 0.1: يكتشف أشخاصاً أكثر، لكن مع أخطاء أكثر (Recall أعلى، Precision أقل).

9. لماذا حوّلنا الصورة من BGR إلى RGB؟

OpenCV يقرأ الألوان بترتيب BGR، والنموذج تعلّم على صور RGB. بدون التحويل يرى النموذج ألواناً خاطئة، فيكتشف أقل (30 بدل 41 شخصاً في تجربتنا).

10. ما الفرق بين yolov5n و yolov5s؟

نفس البنية. الفرق في width_multiple (0.25 مقابل 0.50): s فيه ضعف عدد الفلاتر، فهو أدق لكن أبطأ.

11. لماذا النموذج ضعيف مع الأشخاص البعيدين؟

لأننا استبعدنا من التدريب الأشخاص الأصغر من 2% من الصورة، ولأن حجم 416 فيه تفاصيل أقل. الحل: --img-size 640، أو إعادة التدريب مع الأشخاص الصغار.

12. لماذا نحفظ best.pt وليس آخر جولة فقط؟

لأن النتيجة تتذبذب بين الجولات، وقد تكون آخر جولة أسوأ من جولة سابقة. عندنا: الأفضل كانت الجولة 36 (0.899)، والأخيرة 0.888.